SelVerIR: Self-Verifying Programming Language Intermediate Representation Syntax and Semantics Document v0.8

Yusuf Demir
Yağız Kaan Aydoğdu

March 21, 2026

1 Syntax

$$
\begin{aligned}
\langle IR \rangle &::= \{\langle label \rangle\ \langle Command \rangle\} \\
\langle Command \rangle &::= \texttt{DECL}\ \langle basic\_type \rangle\ ,\langle ident \rangle \mid \texttt{LDECL}\ \langle list\_type \rangle\ ,\langle ident \rangle \\
&\quad \mid \texttt{PUSH}\ \langle imm \rangle \\
&\quad \mid \texttt{LOAD}\ \langle ident \rangle \\
&\quad \mid \texttt{STORE}\ \langle ident \rangle \\
&\quad \mid \texttt{LLOAD}\ \langle ident \rangle \\
&\quad \mid \texttt{LSTORE}\ \langle ident \rangle \\
&\quad \mid \texttt{ADD} \\
&\quad \mid \texttt{SUB} \\
&\quad \mid \texttt{MUL} \\
&\quad \mid \texttt{iDIV} \mid \texttt{fDIV} \\
&\quad \mid \texttt{LEN}\ \langle ident \rangle \\
&\quad \mid \texttt{EQ} \\
&\quad \mid \texttt{LT} \\
&\quad \mid \texttt{LE} \\
&\quad \mid \texttt{GT} \\
&\quad \mid \texttt{GE} \\
&\quad \mid \texttt{NEG} \\
&\quad \mid \texttt{AND} \\
&\quad \mid \texttt{OR} \\
&\quad \mid \texttt{XOR} \\
&\quad \mid \texttt{READ}\ \langle ident \rangle \\
&\quad \mid \texttt{WRITE}\ \langle ident \rangle \\
&\quad \mid \texttt{WRITELN}\ \langle ident \rangle \\
&\quad \mid \texttt{JZ}\ \langle int \rangle \\
&\quad \mid \texttt{GOTO}\ \langle int \rangle \\
&\quad \mid \texttt{CSCOPE} \mid \texttt{PSCOPE} \\
&\quad \mid \texttt{ENV} \\
&\quad \mid \texttt{CALL}\ \langle ident \rangle \\
&\quad \mid \texttt{RET} \\
&\quad \mid \texttt{NOOP} \\
\\
\langle basic\_type \rangle &::= \texttt{INT} \mid \texttt{FLOAT} \\
\langle list\_type \rangle &::= \texttt{LIST[INT]} \mid \texttt{LIST[FLOAT]} \\
\langle ident \rangle &::= [\text{A-Za-z\_}][\text{A-Za-z0-9\_}]^* \\
\langle label \rangle &::= [0\text{-}9]^+\texttt{:} \\
\langle int \rangle &::= [0\text{-}9]^+ \\
\langle float \rangle &::= [0\text{-}9]^+ . [0\text{-}9]^+ \\
\langle imm \rangle &::= \langle int \rangle \mid \langle float \rangle
\end{aligned}
$$

2 Semantics

2.1 Mathematical notions

The `SelVeri` intermediate representation, `SelVerIR` is executed on a *stack-based* abstract machine (**AM**) and aims to be provably correct implementation of the high-level `SelVeri` language.

2.1.1 Stack

At any moment of the execution of a `SelVerIR` program, its stack, denoted as e, is a an ordered finite sequence consisting elements are from **Val**, which will be used in the flow of the program in the *LIFO* order. As expected, e supports a variety of `PUSH` and (indirectly) `POP` operations. For convenience, the empty stack is denoted separately, as ϵ.

2.1.2 Program counter

Intuitively, one can think of any `SelverIR` program, P as an ordered list of commands. Program counter, pc, represents the index of the command that is to be executed next. Control-flow statements such as `GOTO` and `JZ` can alter the program counter in a non-sequential manner.

2.1.3 Return address

A return address, ra, is the index of the command that is to be executed next, just after current function call ends. Instead of storing one ra at any given time, the abstract machine manages a separate stack called ras, containing return addresses in LIFO order, to be able to support nested function calls.

2.1.4 IR-configuration

An IR-configuration is the representation of the execution state in the form of a 6-tuple $\langle c, e, \sigma, s, pc, ras \rangle$ where

- c is the list of commands representing the remaining part of the program `SelverIR` program, i.e, $c = P[pc:]$.
- e is the stack of the execution state,
- σ is the program state,
- s is the scope,
- pc is the program counter.
- ras is the return-address-stack.

2.2 Small-step operational semantics of SelverIR commands

Let P be any SelVerIR program throughout the section **2.2**. Note that any configuration which is not explicitly stated in the tables above leads to the erroneous state, $\hat{\sigma}$ and increments pc by exactly 1.

2.2.1 Small-step operational semantics of state-related commands

$\langle \texttt{DECL}\ t\, x : c,\ e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ e,\ \sigma[x \mapsto \mathbf{0}],\ s[x \mapsto t],\ pc+1 \rangle$ if $x \in \mathbf{Var} \ \wedge\ t \in \{\texttt{INT}, \texttt{FLOAT}\} \ \wedge\ s(x) = \mathbf{Void}$
$\langle \texttt{LDECL}\ t\, x : c,\ n : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ e,\ \sigma[x \mapsto \mathbf{0}],\ s[x \mapsto \texttt{LIST[INT, } n\texttt{]}],\ pc+1 \rangle$ if $x \in \mathbf{Var} \ \wedge\ t = \texttt{LIST[INT]} \ \wedge\ s(x) = \mathbf{Void}$ $\langle c,\ e,\ \sigma[x \mapsto \mathbf{0}],\ s[x \mapsto \texttt{LIST[FLOAT, } n\texttt{]}],\ pc+1 \rangle$ if $x \in \mathbf{Var} \ \wedge\ t = \texttt{LIST[FLOAT]} \ \wedge\ s(x) = \mathbf{Void}$ $\langle c,\ e,\ \hat{\sigma},\ s,\ pc+1 \rangle$ otherwise
$\langle \texttt{PUSH}\ x : c,\ e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ \mathcal{V}(x) : e,\ \sigma,\ s,\ pc+1 \rangle$ if $x \in \mathbf{Imm} \wedge x$ is of basic $\langle c,\ size(\mathbf{List}[t, d, \vec{n}]) : \mathcal{V}(x) : e,\ \sigma,\ s,\ pc+1 \rangle$ if $x \in$
$\langle \texttt{LOAD}\ x : c,\ e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ \sigma(x, s) : e,\ \sigma,\ s,\ pc+1 \rangle$ if $s(x) \in \{\texttt{INT}, \texttt{FLOAT}\} \wedge n = 1 \wedge v_1 \in \mathbb{R} \wedge x \in \mathbf{Var}$ $\langle c,\ n : \sigma(x\texttt{[0]}, s) : \cdots : \sigma(x\texttt{[n-1]}, s) : e,\ \sigma,\ s,\ pc+1 \rangle$ if $s(x) = \texttt{LIST}[t, n] \wedge x \in \mathbf{Var}$ $\langle c,\ e,\ \hat{\sigma},\ s,\ pc+1 \rangle$ otherwise
$\langle \texttt{STORE}\ x : c,\ v_0 : v_1 : \cdots : v_n : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ e,\ \sigma[x \mapsto v_0],\ s,\ pc+1 \rangle$ if $s(x) \in \{\texttt{INT}, \texttt{FLOAT}\} \wedge n = 0 \wedge v_0 \in \mathbb{R} \wedge x \in \mathbf{Var}$ $\langle c,\ e,\ \sigma[x \mapsto [v_1, \ldots, v_n]],\ s,\ pc+1 \rangle$ if $s(x) = \texttt{LIST}[t, v_0] \wedge v_1, \ldots v_n \in \mathbb{R} \wedge x \in \mathbf{Var}$ $\langle c,\ e,\ \hat{\sigma},\ s,\ pc+1 \rangle$ otherwise
$\langle \texttt{LLOAD}\ x : c,\ i : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ \sigma(x\texttt{[}i\texttt{]}, s) : e,\ \sigma,\ s,\ pc+1 \rangle$ if $s(x) = \texttt{LIST}[t, n] \wedge 0 \leq i \leq (n-1) \wedge i, v \in \mathbb{N} \wedge x \in \mathbf{Var}$ $\langle c,\ e,\ \hat{\sigma},\ s,\ pc+1 \rangle$ otherwise
$\langle \texttt{LSTORE}\ x : c,\ i : v : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ e,\ \sigma[x\texttt{[}i\texttt{]} \mapsto v],\ s,\ pc+1 \rangle$ if $s(x) = \texttt{LIST}[t, n] \wedge 0 \leq i \leq (n-1) \wedge i, v \in \mathbb{N} \wedge x \in \mathbf{Var}$ $\langle c,\ e,\ \hat{\sigma},\ s,\ pc+1 \rangle$ otherwise

Table 1: Operational semantics of state-related commands

2.2.2 Small-step operational semantics of arithmetic commands

$\langle \texttt{ADD} : c,\ v_1 : v_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ (v_1 + v_2) : e,\ \sigma,\ s,\ pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \texttt{SUB} : c,\ v_1 : v_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ (v_1 - v_2) : e,\ \sigma,\ s,\ pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \texttt{MUL} : c,\ v_1 : v_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ (v_1 \cdot v_2) : e,\ \sigma,\ s,\ pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \texttt{iDIV} : c,\ z_1 : z_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\begin{cases} \langle c,\ \lfloor z_1 / z_2 \rfloor : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } z_1, z_2 \in \mathbb{Z} \wedge z_2 \neq 0 \\ \langle c,\ \bot : e,\ \hat{\sigma},\ s,\ pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \texttt{fDIV} : c,\ r_1 : r_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\begin{cases} \langle c,\ r_1 / r_2 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } r_1, r_2 \in \mathbb{R} \wedge r_2 \neq 0 \\ \langle c,\ \bot : e,\ \hat{\sigma},\ s,\ pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \texttt{LEN}\ x : c,\ e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\begin{cases} \langle c,\ 0 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } s(x) \in \{\texttt{INT}, \texttt{FLOAT}\} \\ \langle c,\ \mathcal{A}(\_lst\_\texttt{len\_1}, \sigma, s) : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } s(x) = \texttt{LIST[t, } n\texttt{]} \\ \langle c,\ \bot : e,\ \hat{\sigma},\ s,\ pc + 1 \rangle & \text{otherwise} \end{cases}$

Table 2: Operational semantics of arithmetic commands

2.2.3 Small-step operational semantics of boolean commands

$\langle \texttt{EQ} : c,\ v_1 : v_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ (v_1 = v_2) : e,\ \sigma,\ s,\ pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \texttt{LE} : c,\ v_1 : v_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ (v_1 \leq v_2) : e,\ \sigma,\ s,\ pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \texttt{LT} : c,\ v_1 : v_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ (v_1 < v_2) : e,\ \sigma,\ s,\ pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \texttt{GE} : c,\ v_1 : v_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ (v_1 \geq v_2) : e,\ \sigma,\ s,\ pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \texttt{GT} : c,\ v_1 : v_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\langle c,\ (v_1 > v_2) : e,\ \sigma,\ s,\ pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \texttt{AND} : c,\ b_1 : b_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\begin{cases} \langle c,\ 1 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } b_1 = 1 \wedge b_2 = 1 \\ \langle c,\ 0 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } b_1 = 0 \vee b_2 = 0 \end{cases}$
$\langle \texttt{OR} : c,\ b_1 : b_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\begin{cases} \langle c,\ 1 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } b_1 = 1 \vee b_2 = 1 \\ \langle c,\ 0 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } b_1 = 0 \wedge b_2 = 0 \end{cases}$
$\langle \texttt{XOR} : c,\ b_1 : b_2 : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\begin{cases} \langle c,\ 1 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } b_1 \neq b_2 \\ \langle c,\ 0 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } b_1 = b_2 \end{cases}$
$\langle \texttt{NEG} : c,\ b : e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\begin{cases} \langle c,\ 1 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } b = 0 \\ \langle c,\ 0 : e,\ \sigma,\ s,\ pc + 1 \rangle & \text{if } b = 1 \end{cases}$

Table 3: Operational semantics of boolean commands

2.2.4 Small-step operational semantics of I/O commands

$\langle \texttt{READ}\ x : c,\ e,\ \sigma,\ s,\ pc\rangle$	$\triangleright$	$\langle c,\ e,\ \sigma[x \mapsto i],\ s,\ pc+1\rangle$ if i is the input and $x \in \mathbf{Var} \wedge i \in ran\mathcal{L}_{s(x)}$
$\langle \texttt{WRITE}\ x : c,\ e,\ \sigma,\ s,\ pc\rangle$	$\triangleright$	$\langle c,\ e,\ \sigma,\ s,\ pc+1\rangle$ where the abstract machine emits the value $\mathcal{A}(x, \sigma, s)$
$\langle \texttt{WRITELN}\ x : c,\ e,\ \sigma,\ s,\ pc\rangle$	$\triangleright$	$\langle c,\ e,\ \sigma,\ s,\ pc+1\rangle$ where the abstract machine emits the value $\mathcal{A}(x, \sigma, s)$ and a newline

Table 4: Operational semantics of I/O commands

2.2.5 Small-step operational semantics of control-flow commands

$\langle \texttt{JZ}\ z : c,\ b : e,\ \sigma,\ s,\ pc\rangle$	$\triangleright$	$\begin{cases}\langle c,\ e,\ \sigma,\ s,\ pc+1\rangle & \text{if } b = 1 \\ \langle P[\mathcal{I}(z) :],\ e,\ \sigma,\ s,\ \mathcal{I}(z)\rangle & \text{if } b = 0 \ \wedge\ z \in \mathbf{Imm} \cap \mathbf{Int}\end{cases}$
$\langle \texttt{GOTO}\ z : c,\ e,\ \sigma,\ s,\ pc\rangle$	$\triangleright$	$\langle P[\mathcal{I}(z) :],\ e,\ \sigma,\ s,\ \mathcal{I}(z)\rangle$ if $z \in \mathbf{Imm} \cap \mathbf{Int}$
$\langle \texttt{CALL}\ f : c,\ e,\ \sigma,\ s,\ pc,\ ras\rangle$	$\triangleright$	$\langle P[pc_f :],\ e,\ \sigma,\ s,\ pc_f,\ (pc+1) : ras\rangle$ if $f \in \mathbf{Func} \wedge \Gamma(f) \neq \emptyset$
$\langle \texttt{CSCOPE} : c,\ e,\ \sigma,\ s,\ pc\rangle$	$\triangleright$	$\langle c,\ e,\ \tilde{\sigma_c},\ \tilde{s_c},\ pc+1\rangle$ where σ and s are the parents of σ_c and s_c
$\langle \texttt{PSCOPE} : c,\ e,\ \sigma_c,\ s_c,\ pc\rangle$	$\triangleright$	$\langle c,\ e,\ \sigma,\ s,\ pc+1\rangle$ where σ and s are the parents of σ_c and s_c
$\langle \texttt{ENV} : c,\ e,\ \sigma,\ s,\ pc\rangle$	$\triangleright$	$\langle c,\ e,\ \tilde{\sigma}',\ \tilde{s}',\ pc+1\rangle$ where σ and s is of the caller
$\langle \texttt{RET} : c,\ v : e,\ \sigma',\ s',\ pc,\ ra : ras\rangle$	$\triangleright$	$\langle P[ra :],\ e,\ \sigma[\texttt{retvar} \mapsto v],\ s[\texttt{retvar} \mapsto s'(v)],\ ra,\ ras\rangle$ if the re
$\langle \texttt{NOOP} : c,\ e,\ \sigma,\ s,\ pc\rangle$	$\triangleright$	$\langle c,\ e,\ \sigma,\ s,\ pc+1\rangle$
$\langle i : c,\ e,\ \sigma,\ s,\ pc\rangle$	$\triangleright$	$\langle c,\ e,\ \hat{\sigma},\ s,\ pc+1\rangle$ where i is any command

Table 5: Operational semantics of control-flow commands

As one can notice from the tables above, we do not have type-specific commands for `ADD`, `SUB` and `MUL` or boolean operators as these operations are closed under $\mathbb{Z}$ [1]. However, since this is not the case for division, we have separate instructions for integer and float division, namely `iDIV` and `fDIV`.

[1] Although syntactically different in `SelVeri` high level language; semantically, we do not interpret any integer differently than its real number counterpart.

3 Compilation

Let **Code** and **CodeIR** be the sets of all `SelVeri` and `SelVerIR` programs, respectively. Using these sets, we will inductively define a compiler function:

$$\mathcal{C} : \textbf{Code} \to \textbf{CodeIR}$$

3.1 Compilation of lists

Unlike high-level `SelVeri` language, `SelVerIR` does not support multi-dimensional lists, too simplify the maintance of stack during program execution. However, as the lists are interpreted as a 'batch of variables', we may flatten any multi-dimensional list into a one-dimensional list of its basic type. We call this technique *list flattening* and to mathematically formalize it, we define the following function:

$$LF : \textbf{Var} \times \textbf{AExp}^m \to \textbf{AExp}$$

where m is the dimension of the input list variable.

$$LF(lst\,; i_1, \ldots, i_d) = {}^{2}\sum_{j=1}^{d} i_j \left(\prod_{k=j+1}^{d} \texttt{\_lst\_len\_}k \right)$$

LF is obviously well-defined, however we need to prove that LF is injective on indices (i_j) as well to ensure that the compilation is correct.

Proof. Even though not explicitly stated in its formal definition, all i_j represent indices and n_k represent length of the dimension k. As LF will solely be used to flatten a valid indexing of an d-dimensional list, we assume standard 0-based indexing, meaning each index i_k is bounded by $0 \leq i_k < n_k$.

Let $W_j = \prod_{k=j+1}^{d} n_k$ represent the weight of the j-th dimension. Note that $W_m = 1$, and $W_{j-1} = n_j W_j$.

We can rewrite the index calculation as:

$$LF(lst\,; i_1, \ldots, i_d) = i_1 W_1 + i_2 W_2 + \cdots + i_d W_d$$

To prove injectivity, we must show that if $LF(lst\,; i_1, \ldots, i_d) = LF(lst\,; i'_1, \ldots, i'_d)$, then $i_j = i'_j$ for all $1 \leq j \leq d$.

We can prove this by examining the maximum possible value of any suffix sum of the index calculation. The maximum value for the remaining dimensions from $r+1$ to d occurs when every index is at its maximum possible value $(n_j - 1)$:

$$\text{Max Sum} = \sum_{j=r+1}^{d} (n_j - 1) W_j$$

Because $W_{j-1} = n_j W_j$, this sum elegantly telescopes:

$$\text{Max Sum} = ((n_{r+1} - 1)W_{r+1}) + \cdots + ((n_d - 1)W_d)$$

$$\text{Max Sum} = W_r - W_d = W_r - 1$$

This demonstrates a crucial property: the sum of all lower-dimensional components $(i_{r+1}W_{r+1} + \cdots + i_d W_d)$ is strictly less than the weight of the current dimension W_r.

Because the remainder is strictly smaller than the step size of the current dimension, it is impossible for a change in a lower dimension to overflow and alter a higher dimension. Therefore, two different sets of indices must produce different flat indices, proving that LF is injective for d-dimensional lists. □

[2] Because LF constructs an arithmetic expression rather than computing a raw integer, the Σ and Π symbols here denote the syntactic generation of chained + and $*$ expressions

However, LF only formalizes list-accesses in `SelVerIR`, but we must also formalize list declarations in a way compatible with list flattening. Let us define a helper function :

$$\texttt{size\_IR} : \textbf{ListType} \to \textbf{AExp}$$

$$\texttt{size\_IR}(t) = \begin{cases} \texttt{1} & \text{if } t \in \textbf{BasicType} \\ \Pi_{i=1}^{d} a_i & \text{if } t = \textbf{List}[t', d, \vec{a}] \text{ for some } t' \in \textbf{BasicType} \end{cases}$$

Similar to LF, `size_IR` outputs a syntctic arithmetic expression and the use of Π is a shorthan for represeting a chaing of d-many $*$ operations.

Observe that the valuation of $\texttt{size\_IR}(t)$ is equal $size(t)$. With the help of LF and `size_IR`, we will be able to correctly define the compilation of list-accesses and list-declarations in **sections 3.2** and **3.4**.

3.2 Compilation of arithmetic expressions

In order to formally define the compilation of arithmetic expressions, we define a helper function:

$$\mathcal{CA} : \textbf{AExp} \to \textbf{CodeIR}$$

$\mathcal{CA}(x) = \texttt{PUSH}\ x \qquad \text{if } x \in \textbf{Imm}$

$\mathcal{CA}(x) = \texttt{LOAD}\ x \qquad \text{if } x \in \textbf{Var}$

$\mathcal{CA}(lst\texttt{[}i_1\texttt{]} \ldots \texttt{[}i_d\texttt{]}) = \mathcal{CA}(LF(lst; i_1, \ldots, i_d)) : \texttt{LLOAD}\ lst$

if lst is an d-dimensional list

$\mathcal{CA}(\texttt{len(}x\texttt{)}) = \texttt{LEN}\ x$

$\mathcal{CA}(a_1 + a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{ADD}$

$\mathcal{CA}(a_1 * a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{MUL}$

$\mathcal{CA}(a_1 - a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{SUB}$

$$\mathcal{CA}(a_1/a_2) = \begin{cases} \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{iDIV} & \text{if } (a_1/a_2) \text{ has no subterm of type } [3]\textbf{Float} \\ \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{fDIV} & \text{if } (a_1/a_2) \text{ has at least one subterm of type } \textbf{Float} \end{cases}$$

3.3 Compilation of boolean expressions

In order to formally define the compilation of boolean expressions, we define a helper function:

$$\mathcal{CB} : \textbf{BExp} \to \textbf{CodeIR}$$

$\mathcal{CB}(\texttt{true}) = \texttt{PUSH 1}$

$\mathcal{CB}(\texttt{false}) = \texttt{PUSH 0}$

$\mathcal{CB}(a) = \mathcal{CA}(a) : \texttt{PUSH 0} : \texttt{EQ} : \texttt{NEG} \qquad \text{if } a \in \textbf{AExp}$

$\mathcal{CB}(a_1 \texttt{ = } a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{EQ}$

$\mathcal{CB}(a_1 \texttt{ <= } a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{LE}$

$\mathcal{CB}(\texttt{not } b) = \mathcal{CB}(b) : \texttt{NEG}$

$\mathcal{CB}(b_1 \texttt{ and } b_2) = \mathcal{CB}(b_2) : \mathcal{CB}(b_1) : \texttt{AND}$

[3]The types of subterms are determined by the compiler during the compile time.

3.4 Compilation of statements

Finally, with the help of $\mathcal{CA}$ and $\mathcal{CB}$, we are ready to properly define the compilation function:

$$\mathcal{C} : \textbf{Code} \to \textbf{CodeIR}$$

$\mathcal{C}(x : t) = \texttt{DECL}\ t\ x \qquad$ if $x \in \textbf{Var}$ and $t \in \textbf{BasicType}$

$\mathcal{C}(lst : \textbf{List}[t, d, \vec{a}])$

$= \texttt{DECL INT}\ \_lst\_\texttt{len\_1} : \mathcal{CA}(a_1) : \texttt{STORE}\ \_lst\_\texttt{len\_1} : \cdots : \texttt{DECL INT}\ \_lst\_\texttt{len\_}d : \mathcal{CA}(a_d) : \texttt{STORE}\ \_lst\_\texttt{len\_}d$

$: \mathcal{CA}(\texttt{size\_IR}(\textbf{List}[t, d, \vec{a}])) : \texttt{LDECL LIST}[t]\ lst$

$\mathcal{C}(x\ \texttt{:=}\ a) = \mathcal{CA}(a) : \texttt{STORE}\ x \qquad$ if $x \in \textbf{Var}$ and x is of a basic type

$\mathcal{C}(lst[i_1] \ldots [i_d]\ \texttt{:=}\ a) = \mathcal{CA}(a) : \mathcal{CA}(LF(lst; i_1, \ldots, i_d)) : \texttt{LSTORE}\ lst$

if $lst \in \textbf{Var}$ and lst is an d-dimensional list

$\mathcal{C}(\texttt{pass}) = \texttt{NOOP}$

$\mathcal{C}(S_1; S_2) = \mathcal{C}(S_1) : \mathcal{C}(S_2)$

$\mathcal{C}(\texttt{if}\ b\ \texttt{then}\ S\ \texttt{fi}) = \mathcal{CB}(b) : \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{NOOP}$

$\mathcal{C}(\texttt{if}\ b\ \texttt{then}\ S_1\ \texttt{else}\ S_2\ \texttt{fi})$

$= \mathcal{CB}(b) : \texttt{JZ}\ pc_{\texttt{CSCOPE}_2} : \texttt{CSCOPE} : \mathcal{C}(S_1) : \texttt{PSCOPE} : \texttt{GOTO}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP}$

$\mathcal{C}(\texttt{while}\ b\ \texttt{do}\ S\ \texttt{od}) = \mathcal{CB}(b) : \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO}\ pc_b : \texttt{NOOP}$

$\mathcal{C}(\texttt{function}\ f(x_1 : t_1 \ldots, x_n : t_n)\ \texttt{->}\ t_r\ \texttt{::}\ S_f)$

$= \texttt{ENV} : [\texttt{L}]\texttt{DECL}\ t_1\ x_1 : \texttt{STORE}\ x_1 : \cdots : [\texttt{L}]\texttt{DECL}\ t_n\ x_n : \texttt{STORE}\ x_n : \mathcal{C}(S_f)$

$\mathcal{C}(\texttt{call}\ f(a_1 \ldots, a_n)) = \mathcal{CA}(a_n) : \cdots : \mathcal{CA}(a_1) : \texttt{CALL}\ f$

$\mathcal{C}(\texttt{return}\ a) = \mathcal{CA}(a) : \texttt{RET}$

$\mathcal{C}(\texttt{read}\ x) = \texttt{READ}\ x$

$\mathcal{C}(\texttt{write}\ x) = \texttt{WRITE}\ x$

$\mathcal{C}(\texttt{writeline}\ x) = \texttt{WRITELN}\ x$

Please note that, during the compilation phase, compiler labels each compiled command with a corresponding program counter, using a simple incrementing counter. Therefore, *pc* variables seen in the compilation of the control-flow statements are actually replaced with syntactic integer-immediates.

Regarding the in-brackets parts of function definition compilation, a function parameter can be either a list (declared using `LDECL`) or a basic type (declared using `DECL`) To cover this up, our operational semantics ensure that `STORE` also pushes the size of a list to the stack and `LDECL` expects the size of the declared list to be on top of the stack. However, since in both cases (with or without in-brackets parts) the resulting program state and scope tuple is trivially equivalent, we omit this

distinction between `DECL` and `LDECL` in the correctness proof in the next section.

Additionally, as high-level `SelVeri` languagae ensures that every possible execution of S_f ends with a `return`-statement, one can certainly say that $\mathcal{C}(S_f)$ ends with a `RET` command. This fact will be important for our correctess proof and for convenience, it was named as the *return rule*.

4 Provably Correct Implementation

Now that the compilation function is explicitly defined, we need to prove the correctness of our implementation; by showing that given any `SelVeri` program P, we have

$$\langle P,\, \tilde{\sigma},\, \tilde{s}\rangle \;\to^*\; \underline{\sigma}_f \quad\Longrightarrow\quad \langle \mathcal{C}(P),\, \epsilon,\, \tilde{\sigma},\, \tilde{s},\, 0\rangle \;\triangleright^*\; \underline{\sigma}_f$$

4.1 Correctness of expressions

4.1.1 Arithmetic

Given any program stack e, non-erroneous program state σ, scope s, program counter pc, we have:

$$\forall a \in \mathbf{AExp} \cdot \langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle \;\triangleright^*\; \langle c,\, \mathcal{A}(a,\sigma,s) : e,\, \underline{\sigma},\, s,\, pc + l\rangle$$

where l is the number of commands in $\mathcal{CA}(a)$.

Proof. We will prove this by structural induction on arithmetic expressions.

- **case 1:** $a = v$ for some $v \in \mathbf{Imm} \cap (\mathbf{Int} \cup \mathbf{Float})$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(v) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \texttt{PUSH}\ v : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright \langle c,\, \mathcal{V}(v) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(v,\sigma,s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + 1\rangle
\end{aligned}$$

- **case 2:** $a = v$ for some $v \in \mathbf{Imm} \cap (\bigcup \mathbf{ListType})$, i.e. v is a list-immediate of type $\mathbf{List}[t, d, \vec{n}]$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(v) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \texttt{PUSH}\ v : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright \langle c,\, size(\mathbf{List}[t, d, \vec{n}]) : \mathcal{V}(v) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(v,\sigma,s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + 1\rangle
\end{aligned}$$

- **case 3:** $a = x$ for some $x \in \mathbf{Var}$ and x is of a basic type

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(x) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \texttt{LOAD}\ x : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright \langle c,\, \sigma(x,s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(x,\sigma,s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + 1\rangle
\end{aligned}$$

- **case 4:** $a = lst$ for some $lst \in \mathbf{Var}$ and the type of lst is $\mathbf{List}[t, d, \vec{n}]$
  Observe that the size of lst is compiled into $size(\mathbf{List}[t, d, \vec{n}])$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(lst) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \texttt{LOAD}\ lst : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright \langle c,\, size(\mathbf{List}[t, d, \vec{n}]) : \sigma(x, s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(lst, \sigma, s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a, \sigma, s) : e,\, \sigma,\, s,\, pc + 1\rangle
\end{aligned}$$

- **case 5:** $a = lst\texttt{[}i_1\texttt{]} \ldots \texttt{[}i_d\texttt{]}$ for some d-dimensional lst
  Suppose $LF(lst; i_1, \ldots, i_d)) = j$ for some $j \in \mathbf{AExp}$ and assume for an induction on the correctness of the compilation of j.

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(lst\texttt{[}i_1\texttt{]} \ldots \texttt{[}i_d\texttt{]}) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(LF(lst; i_1, \ldots, i_d)) : \texttt{LLOAD}\ lst : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&= \langle \mathcal{CA}(j) : \texttt{LLOAD}\ lst : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(assumption)} \\
&\triangleright^* \langle \texttt{LLOAD}\ lst : c,\, \mathcal{A}(j, \sigma, s) : e,\, \sigma,\, s,\, pc + l_j\rangle && \text{(induction hypothesis)} \\
&\triangleright \langle c,\, \sigma(lst\texttt{[}\mathcal{A}(j, \sigma, s)\texttt{]}, s) : e,\, \sigma,\, s,\, pc + l_j + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \sigma(lst\texttt{[}\mathcal{A}(LF(lst\,; i_1, \ldots, i_d), \sigma, s)\texttt{]}, s) : e,\, \sigma,\, s,\, pc + l_j + 1\rangle && \text{(definition of } LF) \\
&= \langle c,\, \mathcal{A}(lst\texttt{[}\mathcal{A}(i_1, \sigma, s)\texttt{]} \ldots \texttt{[}\mathcal{A}(i_d, \sigma, s)\texttt{]}, \sigma, s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(list flattening)} \\
&= \langle c,\, \mathcal{A}(lst\texttt{[}i_1\texttt{]} \ldots \texttt{[}i_d\texttt{]}, \sigma, s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a, \sigma, s) : e,\, \sigma,\, s,\, pc + 1\rangle
\end{aligned}$$

- **case 6:** $a = \texttt{len(}x\texttt{)}$
  This case is trivially true as `LEN` x exaclty pushes the valuation of $\texttt{len(}x\texttt{)}$ to the stack.
- **case 7:** $a = a_1 + a_2$ for some $a_1, a_2 \in \mathbf{AExp}$

  Assume for an induction on the correctness of the compilation of a_1 and a_2.

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(a_1 + a_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{ADD} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \texttt{ADD} : c,\, \mathcal{A}(a_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_2\rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \texttt{ADD} : c,\, \mathcal{A}(a_1, \sigma, s) : \mathcal{A}(a_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_2 + l_1\rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c,\, (\mathcal{A}(a_1, \sigma, s) + \mathcal{A}(a_2, \sigma, s)) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(a_1 + a_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a, \sigma, s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle
\end{aligned}$$

- **case 8:** $a = a_1 * a_2$ for some $a_1, a_2 \in \mathbf{AExp}$
  Assume for an induction on the correctness of the compilation of a_1 and a_2.

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(a_1 * a_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{MUL} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \texttt{MUL} : c,\, \mathcal{A}(a_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_2\rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \texttt{MUL} : c,\, \mathcal{A}(a_1, \sigma, s) : \mathcal{A}(a_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_2 + l_1\rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c,\, (\mathcal{A}(a_1, \sigma, s) \cdot \mathcal{A}(a_2, \sigma, s)) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(a_1 * a_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a, \sigma, s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle
\end{aligned}$$

- **case 9:** $a = a_1 - a_2$ for some $a_1, a_2 \in$ **AExp**

  Assume for an induction on the correctness of the compilation of a_1 and a_2.

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(a_1 - a_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{SUB} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \texttt{SUB} : c,\, \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2\rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \texttt{SUB} : c,\, \mathcal{A}(a_1,\sigma,s) : \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1\rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c,\, (\mathcal{A}(a_1,\sigma,s) - \mathcal{A}(a_2,\sigma,s)) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(a_1 - a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle
\end{aligned}$$

- **case 10.1:** $a = a_1/a_2$ for some $a_1, a_2 \in$ **AExp** and (a_1/a_2) has no subterm of type **Float**

  Assume for an induction on the correctness of the compilation of a_1 and a_2.

  - **subcase 10.1.1:** $\mathcal{A}(a_2,\sigma,s) \neq 0$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(a_1/a_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{iDIV} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \texttt{iDIV} : c,\, \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2\rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \texttt{iDIV} : c,\, \mathcal{A}(a_1,\sigma,s) : \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1\rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c,\, \lfloor \mathcal{A}(a_1,\sigma,s)/\mathcal{A}(a_2,\sigma,s)\rfloor : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(a_1/a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle
\end{aligned}$$

  - **subcase 10.1.2:** $\mathcal{A}(a_2,\sigma,s) = 0$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(a_1/a_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{iDIV} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \texttt{iDIV} : c,\, \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2\rangle && \text{(induction hypothesis 2)} \\
&= \langle \mathcal{CA}(a_1) : \texttt{iDIV} : c,\, 0 : e,\, \sigma,\, s,\, pc + l_2\rangle \\
&\triangleright^* \langle \texttt{iDIV} : c,\, \mathcal{A}(a_1,\sigma,s) : 0 : e,\, \sigma,\, s,\, pc + l_2 + l_1\rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c,\, \bot : e,\, \hat{\sigma},\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(a_1/a_2,\sigma,s) : e,\, \hat{\sigma},\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a,\sigma,s) : e,\, \hat{\sigma},\, s,\, pc + l_2 + l_1 + 1\rangle
\end{aligned}$$

- **case 10.2:** $a = a_1/a_2$ for some $a_1, a_2 \in$ **AExp** and (a_1/a_2) has at least one subterm of type **Float**

  Assume for an induction on the correctness of the compilation of a_1 and a_2.

  - **subcase 10.2.1:** $\mathcal{A}(a_2,\sigma,s) \neq 0.0$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(a_1/a_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{fDIV} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \texttt{fDIV} : c,\, \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2\rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \texttt{fDIV} : c,\, \mathcal{A}(a_1,\sigma,s) : \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1\rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c,\, \mathcal{A}(a_1,\sigma,s)/\mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(a_1/a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + l_2 + l_1 + 1\rangle
\end{aligned}$$

– **subcase 10.2.2:** $\mathcal{A}(a_2, \sigma, s) = 0.0$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CA}(a_1/a_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{fDIV} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CA}) \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \texttt{fDIV} : c,\, \mathcal{A}(a_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_2\rangle && \text{(induction hypothesis 2)} \\
&= \langle \mathcal{CA}(a_1) : \texttt{fDIV} : c,\, 0.0 : e,\, \sigma,\, s,\, pc + l_2\rangle \\
&\triangleright^* \langle \texttt{fDIV} : c,\, \mathcal{A}(a_1, \sigma, s) : 0.0 : e,\, \sigma,\, s,\, pc + l_2 + l_1\rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c,\, \bot : e,\, \hat{\sigma},\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{A}(a_1/a_2, \sigma, s) : e,\, \hat{\sigma},\, s,\, pc + l_2 + l_1 + 1\rangle && \text{(definition of } \mathcal{A}) \\
&= \langle c,\, \mathcal{A}(a, \sigma, s) : e,\, \hat{\sigma},\, s,\, pc + l_2 + l_1 + 1\rangle
\end{aligned}$$

where l_1 and l_2 are the number of commands in $\mathcal{CA}(a_1)$ and $\mathcal{CA}(a_2)$, respectively.
Observe that, at each case, the increase in pc is exactly equal to the number of commands in $\mathcal{CA}(a)$, as desired.

□

4.1.2 Boolean

Given any program stack e, non-erroneous program state σ, scope s, program counter pc; if b does not have a erroneous subterm:

$$\forall b \in \mathbf{BExp} \cdot \langle \mathcal{CB}(b) : c,\, e,\, \sigma,\, s,\, pc\rangle \triangleright^* \langle c,\, \mathcal{B}(b, \sigma, s) : e,\, \sigma,\, s,\, pc + l\rangle$$

where l is the number of commands in $\mathcal{CB}(b)$.

Proof. We will prove this by an induction on the structure of boolean expressions.

- **case 1:** $b = \texttt{true}$

$$\begin{aligned}
\langle \mathcal{CB}(b) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CB}(\texttt{true}) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \texttt{PUSH 1} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CB}) \\
&\triangleright \langle c,\, \mathcal{V}(\texttt{1}) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, 1 : e,\, \sigma,\, s,\, pc + 1\rangle \\
&= \langle c,\, \mathcal{B}(\texttt{true}, \sigma, s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(definition of } \mathcal{B}) \\
&= \langle c,\, \mathcal{B}(b, \sigma, s) : e,\, \sigma,\, s,\, pc + 1\rangle
\end{aligned}$$

- **case 2:** $b = \texttt{false}$

$$\begin{aligned}
\langle \mathcal{CB}(b) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CB}(\texttt{false}) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \texttt{PUSH 0} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CB}) \\
&\triangleright \langle c,\, \mathcal{V}(\texttt{0}) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, 0 : e,\, \sigma,\, s,\, pc + 1\rangle \\
&= \langle c,\, \mathcal{B}(\texttt{false}, \sigma, s) : e,\, \sigma,\, s,\, pc + 1\rangle && \text{(definition of } \mathcal{B}) \\
&= \langle c,\, \mathcal{B}(b, \sigma, s) : e,\, \sigma,\, s,\, pc + 1\rangle
\end{aligned}$$

- **case 3:** $b \in \mathbf{AExp}$
  - **subcase 3.1:** $\mathcal{A}(a,\sigma,s) \neq 0$

$$\begin{aligned}
\langle \mathcal{CB}(b) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CB}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a) : \texttt{PUSH 0} : \texttt{EQ} : \texttt{NEG} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CB}) \\
&\triangleright^* \langle \texttt{PUSH 0} : \texttt{EQ} : \texttt{NEG} : c,\, \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + l_a\rangle && \text{(correctness of } \mathcal{CA}) \\
&\triangleright \langle \texttt{EQ} : \texttt{NEG} : c,\, 0 : \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + l_a + 1\rangle && \text{(operational semantics)} \\
&\triangleright \langle \texttt{NEG} : c,\, (0 = \mathcal{A}(a,\sigma,s)) : e,\, \sigma,\, s,\, pc + l_a + 2\rangle && \text{(operational semantics)} \\
&= \langle \texttt{NEG} : c,\, 0 : e,\, \sigma,\, s,\, pc + l_a + 2\rangle && \text{(boolean algebra)} \\
&\triangleright \langle c,\, 1 : e,\, \sigma,\, s,\, pc + l_a + 3\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{B}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + l_a + 3\rangle && \text{(definition of } \mathcal{B}) \\
&= \langle c,\, \mathcal{B}(b,\sigma,s) : e,\, \sigma,\, s,\, pc + l_a + 3\rangle
\end{aligned}$$

  - **subcase 3.2:** $\mathcal{A}(a,\sigma,s) = 0$

$$\begin{aligned}
\langle \mathcal{CB}(b) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CB}(a) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a) : \texttt{PUSH 0} : \texttt{EQ} : \texttt{NEG} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CB}) \\
&\triangleright^* \langle \texttt{PUSH 0} : \texttt{EQ} : \texttt{NEG} : c,\, \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + l_a\rangle && \text{(correctness of } \mathcal{CA}) \\
&\triangleright \langle \texttt{EQ} : \texttt{NEG} : c,\, 0 : \mathcal{A}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + l_a + 1\rangle && \text{(operational semantics)} \\
&\triangleright \langle \texttt{NEG} : c,\, (0 = \mathcal{A}(a,\sigma,s)) : e,\, \sigma,\, s,\, pc + l_a + 2\rangle && \text{(operational semantics)} \\
&= \langle \texttt{NEG} : c,\, (0 = 0) : e,\, \sigma,\, s,\, pc + l_a + 2\rangle \\
&= \langle \texttt{NEG} : c,\, 1 : e,\, \sigma,\, s,\, pc + l_a + 2\rangle && \text{(boolean algebra)} \\
&\triangleright \langle c,\, 0 : e,\, \sigma,\, s,\, pc + l_a + 3\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{B}(a,\sigma,s) : e,\, \sigma,\, s,\, pc + l_a + 3\rangle && \text{(definition of } \mathcal{B}) \\
&= \langle c,\, \mathcal{B}(b,\sigma,s) : e,\, \sigma,\, s,\, pc + l_a + 3\rangle
\end{aligned}$$

- **case 4:** $b = (a_1 \texttt{ = } a_2)$ for some $a_1, a_2 \in \mathbf{AExp}$
  - **subcase 4.1:** $\mathcal{A}(a_1,\sigma,s) = \mathcal{A}(a_2,\sigma,s)$

$$\begin{aligned}
\langle \mathcal{CB}(b) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CB}(a_1 \texttt{ = } a_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{EQ} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CB}) \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \texttt{EQ} : c,\, \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_{a_1}\rangle && \text{(correctness of } \mathcal{CA}) \\
&\triangleright^* \langle \texttt{EQ} : c,\, \mathcal{A}(a_1,\sigma,s) : \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2}\rangle && \text{(correctness of } \mathcal{CA}) \\
&\triangleright \langle c,\, (\mathcal{A}(a_1,\sigma,s) = \mathcal{A}(a_2,\sigma,s)) : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2} + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, 1 : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2} + 1\rangle \\
&= \langle c,\, \mathcal{B}(a_1 \texttt{ = } a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2} + 1\rangle && \text{(definition of } \mathcal{B}) \\
&= \langle c,\, \mathcal{B}(b,\sigma,s) : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2} + 1\rangle
\end{aligned}$$

  - **subcase 4.1:** $\mathcal{A}(a_1,\sigma,s) \neq \mathcal{A}(a_2,\sigma,s)$

$$\begin{aligned}
\langle \mathcal{CB}(b) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CB}(a_1 \texttt{ = } a_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \texttt{EQ} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CB}) \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \texttt{EQ} : c,\, \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_{a_1}\rangle && \text{(correctness of } \mathcal{CA}) \\
&\triangleright^* \langle \texttt{EQ} : c,\, \mathcal{A}(a_1,\sigma,s) : \mathcal{A}(a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2}\rangle && \text{(correctness of } \mathcal{CA}) \\
&\triangleright \langle c,\, (\mathcal{A}(a_1,\sigma,s) = \mathcal{A}(a_2,\sigma,s)) : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2} + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, 0 : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2} + 1\rangle \\
&= \langle c,\, \mathcal{B}(a_1 \texttt{ = } a_2,\sigma,s) : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2} + 1\rangle && \text{(definition of } \mathcal{B}) \\
&= \langle c,\, \mathcal{B}(b,\sigma,s) : e,\, \sigma,\, s,\, pc + l_{a_1} + l_{a_2} + 1\rangle
\end{aligned}$$

- **case 5:** $b = (\texttt{not}\ b')$ for some $b' \in \mathbf{BExp}$

  Assume for an induction on the correctness of the compilation of b'.

$$\begin{aligned}
\langle \mathcal{CB}(b) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CB}(\texttt{not}\ b') : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CB}(b') : \texttt{NEG} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CB}) \\
&\triangleright^* \langle \texttt{NEG} : c,\, \mathcal{B}(b', \sigma, s) : e,\, \sigma,\, s,\, pc + l_{b'}\rangle && \text{(induction hypothesis)} \\
&\triangleright \langle c,\, \neg\mathcal{B}(b', \sigma, s) : e,\, \sigma,\, s,\, pc + l_{b'} + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{B}(\texttt{not}\ b', \sigma, s) : e,\, \sigma,\, s,\, pc + l_{b'} + 1\rangle && \text{(definition of } \mathcal{B}) \\
&= \langle c,\, \mathcal{B}(b, \sigma, s) : e,\, \sigma,\, s,\, pc + l_{b'} + 1\rangle
\end{aligned}$$

- **case 6:** $b = (b_1\ \texttt{and}\ b_2)$ for some $b_1, b_2 \in \mathbf{BExp}$

  Assume for an induction on the correctness of the compilations of b_1 and b_2.

$$\begin{aligned}
\langle \mathcal{CB}(b) : c,\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{CB}(b_1\ \texttt{and}\ b_2) : c,\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CB}(b_2) : \mathcal{CB}(b_1) : \texttt{AND} : c,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{CB}) \\
&\triangleright^* \langle \mathcal{CB}(b_1) : \texttt{AND} : c,\, \mathcal{B}(b_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_{b_2}\rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \texttt{AND} : c,\, \mathcal{B}(b_1, \sigma, s) : \mathcal{B}(b_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_{b_2} + l_{b_1}\rangle && \text{(induction hypothesis 2)} \\
&\triangleright \langle c,\, (\mathcal{B}(b_1, \sigma, s) \wedge \mathcal{B}(b_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_{b_2} + l_{b_1} + 1\rangle && \text{(operational semantics)} \\
&= \langle c,\, \mathcal{B}(b_1\ \texttt{and}\ b_2, \sigma, s) : e,\, \sigma,\, s,\, pc + l_{b_2} + l_{b_1} + 1\rangle && \text{(definition of } \mathcal{B}) \\
&= \langle c,\, \mathcal{B}(b, \sigma, s) : e,\, \sigma,\, s,\, pc + l_{b_2} + l_{b_1} + 1\rangle
\end{aligned}$$

where $l_{a'}$ denotes the number of commands in $\mathcal{CA}(a')$ for all $a' \in \mathbf{AExp}$ and $l_{b'}$ denotes the number of commands in $\mathcal{CB}(b')$ for all $b' \in \mathbf{BExp}$.
Observe that in each case the increase in pc is exactly equal to the number of commands in $\mathcal{CB}(b)$, as desired.

□

4.2 Correctness of the statements

Now, as the correctness of arithmetic and boolean expressions is proven, we may restate and prove the correctness of our compiler:

$$\langle P,\, \underline{\sigma},\, s\rangle \ \rightarrow^*\ \langle \underline{\sigma}_f,\, s_f\rangle \quad \Longrightarrow \quad \langle \mathcal{C}(P),\, e,\, \underline{\sigma},\, s,\, pc,\, ras\rangle \ \triangleright^*\ \langle \underline{\sigma}_f,\, s_f,\, pc + l_P,\, ras\rangle$$

where P is any `SelVeri` program and l_P is the number of commands in $\mathcal{C}(P)$.

Observe that the implication above mentions the correctness of only terminating programs. However, we also claim that the compilation of non-terminating `SelVeri` programs are also correct, and we will present our arguments in **section 5**.

Proof. If $\underline{\sigma}$ is an erroneous program state, then by section **2.2.5** of this document and the operational semantic rule of high-level `SelVeri` language [err_{os}], the above implication is obviously true [4]. Therefore, from now on, we assume that $\underline{\sigma}$ represents a non-erroneous program state and will use σ to denote it.

We will prove the correctness of $\mathcal{C}$ via a strong induction on the number of execution steps, by utilizing the fact that sub-statements naturally require a lesser number of execution steps. The correctness of basic type declarations, I/O and `pass` statements establish our base cases, since their compilation include only one command.

[4] Furthermore, $\sigma = \sigma_f$ and $s = s_f$, in this case.

- **(base) case 1:** $P = (x : t)$
  - **subcase 1.1:** $t \in \textbf{BasicType} \wedge s(x) = \textbf{Void}$

$$\begin{aligned}\langle P,\, \sigma,\, s\rangle &= \langle x : t,\, \sigma,\, s\rangle \\ &\to \langle \sigma[x \mapsto \mathbf{0}],\, s[x \mapsto t]\rangle \quad [\text{decl}_{\text{os}}]\end{aligned}$$

    and

$$\begin{aligned}\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(x : t),\, e,\, \sigma,\, s,\, pc\rangle \\ &= \langle \texttt{DECL t},\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{C}) \\ &= \langle \epsilon,\, e,\, \sigma[x \mapsto \mathbf{0}],\, s[x \mapsto t],\, pc+1\rangle && \text{(operational semantics)}\end{aligned}$$

  - **subcase 1.2:** $t \notin \textbf{BasicType} \vee s(x) \neq \textbf{Void}$

$$\begin{aligned}\langle P,\, \sigma,\, s\rangle &= \langle x : t,\, \sigma,\, s\rangle \\ &\to \hat{\sigma} && [\text{decl}^{\perp}_{\text{os}}]\end{aligned}$$

    and

$$\begin{aligned}\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(x : t),\, e,\, \sigma,\, s,\, pc\rangle \\ &= \langle \texttt{DECL t},\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{C}) \\ &= \langle \epsilon,\, e,\, \hat{\sigma},\, s,\, pc+1\rangle && \text{(operational semantics)}\end{aligned}$$

- **(base) case 2:** $P = \texttt{pass}$

$$\begin{aligned}\langle P,\, \sigma,\, s\rangle &= \langle \texttt{pass},\, \sigma,\, s\rangle \\ &\to \langle \sigma,\, s\rangle && [\text{pass}_{\text{os}}]\end{aligned}$$

  and

$$\begin{aligned}\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(\texttt{pass}),\, e,\, \sigma,\, s,\, pc\rangle \\ &= \langle \texttt{NOOP},\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{C}) \\ &= \langle \epsilon,\, e,\, \sigma,\, s,\, pc+1\rangle && \text{(operational semantics)}\end{aligned}$$

- **(base) case 3:** P is an IO-statement, i.e. $P \in \{\texttt{read } x, \texttt{write } x, \texttt{writeline } x\}$. Then, the compilation of IO-statement is trivially true as they compile into one IR-command whose semantics apply the exact same change to the program state and scope.
- **case 4:** $P = (lst : \textbf{List}[t, d, \vec{a}])$

  Let $\sigma' = \sigma[\_lst\_\texttt{len\_i} \mapsto \mathcal{A}(a_i, \sigma, s)]$ and $s' = s[\_lst\_\texttt{len\_i} \mapsto \textbf{Int}]$ for $i \in \{1, \ldots, d\}$

$$\begin{aligned}\langle P,\, \sigma,\, s\rangle &= \langle lst : \textbf{List}[t, d, \vec{a}],\, \sigma,\, s\rangle \\ &\to \langle \sigma'[lst \mapsto \mathbf{0}], s'[lst \mapsto \textbf{List}[t, d, \mathcal{A}(\vec{a_i}, \sigma, s)]]\rangle \quad [\text{decl}^{\text{list}}_{\text{os}}]\end{aligned}$$

  and

$$\begin{aligned}\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(lst : \textbf{List}[t, a]),\, e,\, \sigma,\, s,\, pc\rangle \\ &= \langle \texttt{DECL INT } \_lst\_\texttt{len\_1} : \mathcal{CA}(a_1) : \texttt{STORE } \_lst\_\texttt{len\_1} : \cdots : \texttt{DECL INT } \_lst\_\texttt{len\_}d : \mathcal{CA}(a_d) : \texttt{STORE } \_ \\ &\triangleright^{*} \langle \mathcal{CA}(\texttt{size\_IR(}\textbf{List}[t, a]\texttt{)}) : \texttt{LDECL LIST[}t\texttt{]}\ lst,\, e,\, \sigma',\, s',\, pc+l\rangle \\ &\triangleright \langle \texttt{LDECL LIST[}t\texttt{]}\ lst,\, \mathcal{A}(\texttt{size\_IR(}\textbf{List}[t, d, \vec{a}]\texttt{)}, \sigma, s) : e,\, \sigma[\_lst\_\texttt{len\_1} \mapsto \mathcal{A}(a, \sigma, s)],\, s[\_lst\_\texttt{len\_1} \mapsto \\ &\triangleright \langle \epsilon,\, e,\, \sigma'[lst \mapsto \mathbf{0}],\, s'[lst \mapsto \texttt{LIST[}t, \mathcal{A}(\texttt{size\_IR(List[}t, d, \vec{a}\texttt{])}, \sigma, s)\texttt{]}],\, pc+l+l_a+1\rangle \\ &= \langle \epsilon,\, e,\, \sigma'[lst \mapsto \mathbf{0}],\, s'[lst \mapsto \textbf{List}[t, d, \mathcal{A}(\vec{a_i}, \sigma, s)]],\, pc+l+l_a+2\rangle\end{aligned}$$

Note that, for all use cases of $\mathcal{A}$ in the proof above, the valuation does not change whether σ, s or σ', s' is used, as only changes between two tuples are related to newly declared variables.

- **case 5:** $P = (x \texttt{ := } a)$

  Assume that x is of a basic type and $\mathcal{A}(a, \sigma, s) \neq \bot \wedge \mathcal{A}(a, \sigma, s) \in ran\mathcal{L}_{s(x)}$, then:

$$\begin{aligned}\langle P,\, \sigma,\, s\rangle &= \langle x \texttt{ := } a,\, \sigma,\, s\rangle \\ &\quad\ \sigma[x \mapsto \mathcal{A}(a, \sigma, s)] \quad [\mathrm{ass}_{\mathrm{os}}]\end{aligned}$$

  then

$$\begin{aligned}\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(x \texttt{ := } a),\, e,\, \sigma,\, s,\, pc\rangle \\ &= \langle \mathcal{CA}(a) : \texttt{STORE } x,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{C}) \\ &\triangleright^* \langle \texttt{STORE } x,\, \mathcal{A}(a, \sigma, s) : e,\, \sigma,\, s,\, pc + l_a\rangle && \text{(correctness of } \mathcal{CA}) \\ &= \langle \epsilon,\, e,\, \sigma[x \mapsto \mathcal{A}(a, \sigma, s)],\, s,\, pc + l_a + 1\rangle && \text{(operational semantics)}\end{aligned}$$

- **case 6:** $P = (lst\texttt{[}i_1\texttt{]} \ldots \texttt{[}i_d\texttt{]} \texttt{ := } a)$

  Assume that lst is an d-dimensional list and $\mathcal{A}(a, \sigma, s) \neq \bot \wedge \mathcal{A}(a, \sigma, s) \in ran\mathcal{L}_{s(x)}$, then:

$$\begin{aligned}\langle P,\, \sigma,\, s\rangle &= \langle lst\texttt{[}i_1\texttt{]} \ldots \texttt{[}i_d\texttt{]} \texttt{ := } a,\, \sigma,\, s\rangle \\ &\quad\ \sigma[lst\texttt{[}\mathcal{A}(i_1, \sigma, s)\texttt{]} \ldots \texttt{[}\mathcal{A}(i_d, \sigma, s)\texttt{]} \mapsto \mathcal{A}(a, \sigma, s)] \quad [\mathrm{ass}^{\mathrm{list}}_{\mathrm{os}}]\end{aligned}$$

  then

$$\begin{aligned}\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(lst\texttt{[}i_1\texttt{]} \ldots \texttt{[}i_d\texttt{]} \texttt{ := } a),\, e,\, \sigma,\, s,\, pc\rangle \\ &= \langle \mathcal{CA}(a) : \mathcal{CA}(LF(lst; i_1, \ldots, i_d)) : \texttt{LSTORE } lst,\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{C}) \\ &\triangleright^* \langle \mathcal{CA}(LF(lst; i_1, \ldots, i_d)) : \texttt{LSTORE } lst,\, \mathcal{A}(a, \sigma, s) : e,\, \sigma,\, s,\, pc + l_a\rangle && \text{(correctness of } \mathcal{CA}) \\ &\triangleright^* \langle \texttt{LSTORE } lst,\, \mathcal{A}(LF(lst\,; i_1, \ldots, i_d), \sigma, s) : \mathcal{A}(a, \sigma, s) : e,\, \sigma,\, s,\, pc + l_a + l_I\rangle && \text{(correctness of } \mathcal{CA}) \\ &\triangleright \langle \epsilon,\, e,\, \sigma[lst\texttt{[}\mathcal{A}(LF(lst\,; i_1, \ldots, i_d), \sigma, s)\texttt{]} \mapsto \mathcal{A}(a, \sigma, s)],\, s,\, pc + l_a + l_I + 1\rangle && \text{(operational semanti} \\ &= \langle \epsilon,\, e,\, \sigma[lst\texttt{[}\mathcal{A}(i_1, \sigma, s)\texttt{]} \ldots \texttt{[}\mathcal{A}(i_d, \sigma, s)\texttt{]} \mapsto \mathcal{A}(a, \sigma, s)],\, s,\, pc + l_a + l_I + 1\rangle && \text{(list flattening)}\end{aligned}$$

- **case 7:** $P = (\texttt{if } b \texttt{ then } S \texttt{ fi})$

  - **subcase 7.1:** $\mathcal{B}(b, \sigma, s) = 1$
    Assume

$$\begin{aligned}\langle P,\, \sigma,\, s\rangle &= \langle \texttt{if } b \texttt{ then } S \texttt{ fi},\, \sigma,\, s\rangle \\ &= \langle \texttt{if } b \texttt{ then } S \texttt{ else pass fi},\, \sigma,\, s\rangle && \text{(semantical equivalance)} \\ &\to \langle S\texttt{;endscope},\, \tilde{\sigma_c},\, \tilde{s_c}\rangle && [\mathrm{if}^1_{\mathrm{os}}] \\ &\to^* \langle \texttt{endscope},\, \sigma'_c,\, s'_c\rangle && ([\mathrm{comp}^1_{\mathrm{os}}]\,, [\mathrm{comp}^2_{\mathrm{os}}]) \\ &\to \langle \underline{\sigma}',\, s'\rangle && [\mathrm{scope}_{\mathrm{os}}]\end{aligned}$$

    and assume for an induction hypothesis, that is, the compilation of S is correct, then

$$
\begin{aligned}
\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(\texttt{if}\ b\ \texttt{then}\ S\ \texttt{fi}),\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CB}(b) : \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{NOOP},\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{C}) \\
&\triangleright^* \langle \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{NOOP},\, \mathcal{B}(b,\sigma,s) : e,\, \sigma,\, s,\, pc + l_b\rangle && \text{(correctness of } \mathcal{CB}) \\
&= \langle \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{NOOP},\, 1 : e,\, \sigma,\, s,\, pc + l_b\rangle \\
&\triangleright \langle \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{NOOP},\, e,\, \sigma,\, s,\, pc + l_b + 1\rangle && \text{(operational seman} \\
&\triangleright \langle \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{NOOP},\, e,\, \tilde{\sigma_c},\, \tilde{s_c},\, pc + l_b + 2\rangle && \text{(operational seman} \\
&\triangleright^* \langle \texttt{PSCOPE} : \texttt{NOOP},\, e',\, \sigma'_c,\, s'_c,\, pc + l_b + l_S + 2\rangle && \text{(induction hypothe} \\
&\triangleright \langle \texttt{NOOP},\, e',\, \underline{\sigma}',\, s',\, pc + l_b + l_S + 3\rangle && \text{(induction hypothe} \\
&\triangleright \langle \epsilon,\, e',\, \underline{\sigma}',\, s',\, pc + l_b + l_S + 4\rangle && \text{(operational seman}
\end{aligned}
$$

– **subcase 7.2:** $\mathcal{B}(b,\sigma,s) = 0$
Assume

$$
\begin{aligned}
\langle P,\, \sigma,\, s\rangle &= \langle \texttt{if}\ b\ \texttt{then}\ S\ \texttt{fi},\, \sigma,\, s\rangle \\
&= \langle \texttt{if}\ b\ \texttt{then}\ S\ \texttt{else pass fi},\, \sigma,\, s\rangle && \text{(semantical equivalance)} \\
&\rightarrow \langle \texttt{pass;endscope},\, \tilde{\sigma_c},\, \tilde{s_c}\rangle && [\text{if}^0_{\text{os}}] \\
&\rightarrow^* \langle \texttt{endscope},\, \tilde{\sigma_c},\, \tilde{s_c}\rangle && ([\text{pass}_{\text{os}}],\, [\text{comp}^1_{\text{os}}],\, [\text{comp}^2_{\text{os}}]) \\
&\rightarrow \langle \sigma,\, s\rangle && [\text{scope}_{\text{os}}]
\end{aligned}
$$

then,

$$
\begin{aligned}
\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(\texttt{if}\ b\ \texttt{then}\ S\ \texttt{fi}),\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CB}(b) : \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{NOOP},\, e,\, \sigma,\, s,\, pc\rangle && \text{(definition of } \mathcal{C}) \\
&\triangleright^* \langle \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{NOOP},\, \mathcal{B}(b,\sigma,s) : e,\, \sigma,\, s,\, pc + l_b\rangle && \text{(correctness of } \mathcal{CB}) \\
&= \langle \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{NOOP},\, 0 : e,\, \sigma,\, s,\, pc + l_b\rangle \\
&\triangleright \langle \texttt{NOOP},\, e,\, \sigma,\, s,\, pc_{\texttt{NOOP}}\rangle && \text{(operational seman} \\
&\triangleright \langle \epsilon,\, e,\, \sigma,\, s,\, pc_{\texttt{NOOP}} + 1\rangle && \text{(operational seman}
\end{aligned}
$$

- **case 8:** $P = (\texttt{if}\ b\ \texttt{then}\ S_1\ \texttt{else}\ S_2\ \texttt{fi})$

– **subcase 8.1:** $\mathcal{B}(b,\sigma,s) = 1$
Assume

$$
\begin{aligned}
\langle P,\, \sigma,\, s\rangle &= \langle \texttt{if}\ b\ \texttt{then}\ S_1\ \texttt{else}\ S_2\ \texttt{fi},\, \sigma,\, s\rangle \\
&\rightarrow \langle S_1\texttt{;endscope},\, \tilde{\sigma_c},\, \tilde{s_c}\rangle && [\text{if}^1_{\text{os}}] \\
&\rightarrow^* \langle \texttt{endscope},\, \sigma'_c,\, s'_c\rangle && ([\text{comp}^1_{\text{os}}],\, [\text{comp}^2_{\text{os}}]) \\
&\rightarrow \langle \underline{\sigma}',\, s'\rangle && [\text{scope}_{\text{os}}]
\end{aligned}
$$

and assume for an induction hypothesis, that is, the compilation of S_1 is correct, then

$$\begin{aligned}
\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(\texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2 \texttt{ fi}),\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{C}\mathcal{B}(b) : \texttt{JZ } pc_{\texttt{CSCOPE}_2} : \texttt{CSCOPE} : \mathcal{C}(S_1) : \texttt{PSCOPE} : \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOO} \\
&\triangleright^* \langle \texttt{JZ } pc_{\texttt{CSCOPE}_2} : \texttt{CSCOPE} : \mathcal{C}(S_1) : \texttt{PSCOPE} : \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, \mathcal{B}(b, \\
&= \langle \texttt{JZ } pc_{\texttt{CSCOPE}_2} : \texttt{CSCOPE} : \mathcal{C}(S_1) : \texttt{PSCOPE} : \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, 1 : e, \\
&\triangleright \langle \texttt{CSCOPE} : \mathcal{C}(S_1) : \texttt{PSCOPE} : \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, e,\, \sigma,\, s,\, pc + l_b + 1\rangle \\
&\triangleright \langle \mathcal{C}(S_1) : \texttt{PSCOPE} : \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, e,\, \tilde{\sigma_c},\, \tilde{s_c},\, pc + l_b + 2\rangle \\
&\triangleright^* \langle \texttt{PSCOPE} : \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, e',\, \sigma_c',\, s_c',\, pc + l_b + l_{S_1} + 2\rangle \\
&\triangleright \langle \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, e',\, \underline{\sigma}',\, s',\, pc + l_b + l_{S_1} + 3\rangle \\
&\triangleright \langle \texttt{NOOP},\, e',\, \underline{\sigma}',\, s',\, pc_{\texttt{NOOP}}\rangle \\
&\triangleright \langle \epsilon,\, e',\, \underline{\sigma}',\, s',\, pc_{\texttt{NOOP}} + 1\rangle
\end{aligned}$$

– **subcase 8.2:** $\mathcal{B}(b, \sigma, s) = 0$
Assume

$$\begin{aligned}
\langle P,\, \sigma,\, s\rangle &= \langle \texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2 \texttt{ fi},\, \sigma,\, s\rangle \\
&\to \langle S_2 \texttt{;endscope},\, \tilde{\sigma_c},\, \tilde{s_c}\rangle && [\text{if}^1_{\text{os}}] \\
&\to^* \langle \texttt{endscope},\, \sigma_c',\, s_c'\rangle && ([\text{comp}^1_{\text{os}}],\, [\text{comp}^2_{\text{os}}]) \\
&\to \langle \underline{\sigma}',\, s'\rangle && [\text{scope}_{\text{os}}]
\end{aligned}$$

and assume for an induction hypothesis, that is, the compilation of S_2 is correct, then

$$\begin{aligned}
\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(\texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2 \texttt{ fi}),\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{C}\mathcal{B}(b) : \texttt{JZ } pc_{\texttt{CSCOPE}_2} : \texttt{CSCOPE} : \mathcal{C}(S_1) : \texttt{PSCOPE} : \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOO} \\
&\triangleright^* \langle \texttt{JZ } pc_{\texttt{CSCOPE}_2} : \texttt{CSCOPE} : \mathcal{C}(S_1) : \texttt{PSCOPE} : \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, \mathcal{B}(b, \\
&= \langle \texttt{JZ } pc_{\texttt{CSCOPE}_2} : \texttt{CSCOPE} : \mathcal{C}(S_1) : \texttt{PSCOPE} : \texttt{GOTO } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, 0 : e, \\
&\triangleright \langle \texttt{CSCOPE} : \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, e,\, \sigma,\, s,\, pc_{\texttt{CSCOPE}_2}\rangle \\
&\triangleright \langle \mathcal{C}(S_2) : \texttt{PSCOPE} : \texttt{NOOP},\, e,\, \tilde{\sigma_c},\, \tilde{s_c},\, pc_{\texttt{CSCOPE}_2} + 1\rangle \\
&\triangleright^* \langle \texttt{PSCOPE} : \texttt{NOOP},\, e',\, \underline{\sigma_c}',\, s_c',\, pc_{\texttt{CSCOPE}_2} + l_{S_2} + 1\rangle \\
&\triangleright \langle \texttt{NOOP},\, e',\, \underline{\sigma}',\, s',\, pc_{\texttt{CSCOPE}_2} + l_{S_2} + 2\rangle \\
&\triangleright \langle \epsilon,\, e',\, \underline{\sigma}',\, s',\, pc_{\texttt{CSCOPE}_2} + l_{S_2} + 3\rangle
\end{aligned}$$

- **case 9:** $P = (\texttt{while } b \texttt{ do } S \texttt{ od})$

– **subcase 9.1:** $\mathcal{B}(b, \sigma, s) = 0$
Assume

$$\begin{aligned}
\langle P,\, \sigma,\, s\rangle &= \langle \texttt{while } b \texttt{ do } S \texttt{ od},\, \sigma,\, s\rangle \\
&\to \langle \sigma,\, s\rangle && [\text{while}^0_{\text{os}}]
\end{aligned}$$

then,

$$\begin{aligned}
\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(\texttt{while } b \texttt{ do } S \texttt{ od}),\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{C}\mathcal{B}(b) : \texttt{JZ } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO } pc_b : \texttt{NOOP},\, e,\, \sigma,\, s,\, pc\rangle && \text{(definitio} \\
&\triangleright^* \langle \texttt{JZ } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO } pc_b : \texttt{NOOP},\, \mathcal{B}(b, \sigma, s) : e,\, \sigma,\, s,\, pc + l_b\rangle && \text{(correctn} \\
&= \langle \texttt{JZ } pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO } pc_b : \texttt{NOOP},\, 0 : e,\, \sigma,\, s,\, pc + l_b\rangle \\
&\triangleright \langle \texttt{NOOP},\, e,\, \sigma,\, s,,\, pc_{\texttt{NOOP}}\rangle && \text{(operatio} \\
&\triangleright \langle \epsilon,\, e,\, \sigma,\, s,,\, pc_{\texttt{NOOP}} + 1\rangle && \text{(operatio}
\end{aligned}$$

– **subcase 9.2:** $\mathcal{B}(b, \sigma, s) = 1$

$$
\begin{aligned}
\langle P,\, \sigma,\, s\rangle &= \langle \texttt{while}\ b\ \texttt{do}\ S\ \texttt{od},\, \sigma,\, s\rangle \\
&\to \langle S\texttt{;endscope;while}\ b\ \texttt{do}\ S\ \texttt{od},\, \tilde{\sigma_c},\, \tilde{s_c}\rangle && [\text{while}^1_{\text{os}}] \\
&\to^* \langle \texttt{endscope;while}\ b\ \texttt{do}\ S\ \texttt{od},\, \sigma'_c,\, s'_c\rangle && ([\text{comp}^1_{\text{os}}],\, [\text{comp}^2_{\text{os}}]) \\
&\to \langle \texttt{while}\ b\ \texttt{do}\ S\ \texttt{od},\, \underline{\sigma}',\, s'\rangle && ([\text{scope}_{\text{os}}],\, [\text{comp}^1_{\text{os}}],\, [\text{comp}^2_{\text{os}}])
\end{aligned}
$$

and assume for an induction hypothesis, that is, the compilation of S is correct, then

$$
\begin{aligned}
\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(\texttt{while}\ b\ \texttt{do}\ S\ \texttt{od}),\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{CB}(b) : \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO}\ pc_b : \texttt{NOOP},\, e,\, \sigma,\, s,\, pc\rangle && \text{(definitio} \\
&\triangleright^* \langle \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO}\ pc_b : \texttt{NOOP},\, \mathcal{B}(b, \sigma, s) : e,\, \sigma,\, s,\, pc + l_b\rangle && \text{(correctn} \\
&= \langle \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO}\ pc_b : \texttt{NOOP},\, 1 : e,\, \sigma,\, s,\, pc + l_b\rangle \\
&\triangleright \langle \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO}\ pc_b : \texttt{NOOP},\, 1 : e,\, \sigma,\, s,\, pc + l_b + 1\rangle && \text{(operatio} \\
&\triangleright \langle \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO}\ pc_b : \texttt{NOOP},\, e,\, \tilde{\sigma_c},\, \tilde{s_c},\, pc + l_b + 2\rangle && \text{(operatio} \\
&\triangleright^* \langle \texttt{PSCOPE} : \texttt{GOTO}\ pc_b : \texttt{NOOP},\, e',\, \sigma'_c,\, s'_c,\, pc + l_b + l_S + 2\rangle && \text{(inductio} \\
&\triangleright \langle \texttt{GOTO}\ pc_b : \texttt{NOOP},\, e',\, \underline{\sigma}',\, s',\, pc + l_b + l_S + 3\rangle && \text{(operatio} \\
&\triangleright \langle \mathcal{CB}(b) : \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO}\ pc_b : \texttt{NOOP},\, e',\, \underline{\sigma}',\, s',\, pc_b\rangle && \text{(operatio} \\
&= \langle \mathcal{CB}(b) : \texttt{JZ}\ pc_{\texttt{NOOP}} : \texttt{CSCOPE} : \mathcal{C}(S) : \texttt{PSCOPE} : \texttt{GOTO}\ pc_b : \texttt{NOOP},\, e',\, \underline{\sigma}',\, s',\, pc\rangle \\
&= \langle \mathcal{C}(\texttt{while}\ b\ \texttt{do}\ S\ \texttt{od}),\, e',\, \underline{\sigma}',\, s',\, pc\rangle
\end{aligned}
$$

However, the proofs above are not enough to establish the correctness of the compiled `while`-statements, because they only demonstrate one step of iteration. Nonetheless, with a simple induction, we can generalize this argument to claim that at any step of iteration, the compilation is correct, as the proofs above show that at any given program state and scope, the compilation of one iteration is correct. To put it more explicitly, the cases of 0 iterations (**subcase 6.1**) and 1 iteration (**subcase 6.2**) are proven to be correct. Assuming that i iterations are compiled correctly, **subcase 6.2** also prove that $i + 1$ iterations are compiled correctly. Ultimately, one can certainly say that any terminating `SelVeri` loop is compiled correctly.

- **case 10:** $P = \texttt{call}\ f(a_1 \ldots, a_n)$

  Assume that $s'(x_i) = t_i$ and $\sigma'(x_i) = \mathcal{A}(a_i, \sigma, s)$ for $i = 1, 2, \ldots, n$

$$
\begin{aligned}
\langle P,\, \sigma,\, s\rangle &= \langle \texttt{call}\ f(a_1 \ldots, a_n),\, \sigma,\, s\rangle \\
&\to \langle x_1 : t_1\texttt{;}x_1\ \texttt{:=}\ v_1\ \texttt{;}\ \ldots\ \texttt{;}x_n : t_n\texttt{;}x_n\ \texttt{:=}\ v_n\ \texttt{;}S_f,\, \tilde{\sigma}',\, \tilde{s}'\rangle && [\text{call}_{\text{os}}] \\
&\to^* \langle S_f,\, \sigma',\, s'\rangle && \text{(operational semantics)} \\
&\to^* \langle \texttt{return}\ a,\, \sigma''^{5},\, s''\rangle && \text{(operational semantics, } \textit{return rule}\text{)} \\
&\to \langle \sigma[\texttt{retvar} \mapsto \mathcal{A}(a, \sigma'', s'')],\ s[\texttt{retvar} \mapsto s''(a)]\rangle && [\text{ret}_{\text{os}}]
\end{aligned}
$$

by the induction hypothesis, the compilation of S_f is correct, since the execution of the function body needs lesser steps than the whole function call; then

$$
\begin{aligned}
\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc,\, ras\rangle &= \langle \mathcal{C}(\texttt{call}\ f(a_1 \ldots, a_n)),\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{C}\mathcal{A}(a_n) : \cdots : \mathcal{C}\mathcal{A}(a_1) : \texttt{CALL}\ f,\, e,\, \sigma,\, s,\, pc,\, ras\rangle \\
&\triangleright^* \langle \texttt{CALL}\ f,\, \mathcal{A}(a_1, \sigma, s) : \cdots : \mathcal{A}(a_n, \sigma, s) : e,\, \sigma,\, s,\, pc + n,\, ras\rangle \\
&\triangleright \langle P[pc_f],\, \mathcal{A}(a_1, \sigma, s) : \cdots : \mathcal{A}(a_n, \sigma, s) : e,\, \sigma,\, s,\, pc_f,\, (pc + n + 1) : ras\rangle \\
&= \langle \texttt{ENV} : \texttt{DECL}\ t_1\ x_1 : \texttt{STORE}\ x_1 : \cdots : \texttt{DECL}\ t_1\ x_1 : \texttt{STORE}\ x_n : \mathcal{C}(S_f),\, \mathcal{A}(a_1, \sigma, s) : \cdots : \mathcal{A}(a_n, \sigma, s \\
&\triangleright \langle \texttt{DECL}\ t_1\ x_1 : \texttt{STORE}\ x_1 : \cdots : \texttt{DECL}\ t_1\ x_1 : \texttt{STORE}\ x_n : \mathcal{C}(S_f),\, \mathcal{A}(a_1, \sigma, s) : \cdots : \mathcal{A}(a_n, \sigma, s) : e,\, \\
&\triangleright^* \langle \mathcal{C}(S_f),\, e,\, \sigma',\, s',\, pc_f + 2n + 1,\, (pc + n + 1) : ras\rangle \\
&\triangleright^* \langle \mathcal{C}(\texttt{return}\ a),\, e,\, \sigma'',\, s'',\, pc_f + l + 2n + 1,\, (pc + n + 1) : ras\rangle \\
&= \langle \mathcal{C}\mathcal{A}(a) : \texttt{RET},\, e,\, \sigma'',\, s'',\, pc_f + l + 2n + 1,\, (pc + n + 1) : ras\rangle \\
&\triangleright^* \langle \texttt{RET},\, \mathcal{A}(a, \sigma'', s'') : e,\, \sigma'',\, s'',\, pc_f + l' + 2n + 1,\, (pc + n + 1) : ras\rangle \\
&\triangleright \langle P[pc + n + 1 :],\, e,\, \sigma[\texttt{retvar} \mapsto \mathcal{A}(a, \sigma'', s'')],\ s[\texttt{retvar} \mapsto s''(a)],\, pc + n + 1,\, ras\rangle
\end{aligned}
$$

Similar to our proof of the correctness of `while`-statements, this assumes a finite depth of recursion (0, if there is no recursion) as we have assumed termination.

- **case 11:** $P = S_1\,;S_2$ for some statements S_1, S_2

  Assume

$$
\begin{aligned}
\langle P,\, \sigma,\, s\rangle &= \langle S_1\,;S_2,\, \sigma,\, s\rangle \\
&\rightarrow^* \langle S_2,\, \underline{\sigma}',\, s'\rangle \qquad ([\text{comp}^1_{\text{os}}]\,,\, [\text{comp}^2_{\text{os}}]) \\
&\rightarrow^* \langle \underline{\sigma}'',\, s''\rangle \qquad ([\text{comp}^1_{\text{os}}]\,,\, [\text{comp}^2_{\text{os}}])
\end{aligned}
$$

  and, assume for an induction, that is the compilation of S_1 and S_2 are correct, then

$$
\begin{aligned}
\langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc\rangle &= \langle \mathcal{C}(S_1\,;S_2),\, e,\, \sigma,\, s,\, pc\rangle \\
&= \langle \mathcal{C}(S_1) : \mathcal{C}(S_2),\, e,\, \sigma,\, s,\, pc\rangle \qquad \text{(definition of } \mathcal{C}) \\
&= \langle \mathcal{C}(S_2),\, e',\, \underline{\sigma}',\, s',\, pc + l_{S_1}\rangle \qquad \text{(induction hypothesis 1)} \\
&= \langle \epsilon,\, e'',\, \underline{\sigma}'',\, s'',\, pc + l_{S_1} + l_{S_2}\rangle \qquad \text{(induction hypothesis 2)}
\end{aligned}
$$

Observe that at each case, the resulting program state and scope tuple is the equal for `SelVeri` and `SelVerIR`, the difference between initial and final program counters are equal to l_p and ras is not altered, therefore concluding [6] the proof.

□

[5] Note that we do not cover the case where the execution of S_f leads to an erroneous state, as the last semantic rule in the section **2.2.5** of this document and [err_{os}] rule of high-level semantics ensure the correctness of the translation in this case.

[6] Note that we need not prove the correctness of `return`-statements separately; as they are always a part of function calls, case 7 also establishes the correctness of `return`-statements

Cautious readers may notice that some (sub)cases including erroneous program states (either as input or output) is omitted in our correctness proofs. One of the following reasons is the justification for this omittance:

1. Such a case cannot be encountered in the range of neither $\mathcal{CA}$ nor $\mathcal{CB}$ nor $\mathcal{C}$.
2. Such an erroneous case can be detected during compile-time, therefore no `SelVerIR` code inheriting the error can be generated.
3. After a finite number of non-erroneous execution steps, which are proven correct in this section, program reaches an error state; but by section **2.2.5** of this document and the operational semantic rule of high-level `SelVeri` language [err_{os}] the remaining finite execution is also correct.

No matter the reason, as no statically-correct `SelVeri` program includes any of the omitted cases, the correctness proofs above are sufficient for establishing the correctness of our implementation.

5 Divergent programs

If a `SelVeri` program P terminates after a finite number a steps, it is called *convergent*; otherwise, a non-terminating P is called *divergent*. `SelVeri` has two possible conditions for non-termination:

1. Infinite `while`-loops
2. Infinite recursion

Therefore any infinite program, after the execution of a finite subprogram, is an infinite repetition of a (possibly compositional) statement, which is either a loop body, function body or a composition of function bodies; whose execution takes a finite number of steps. We call this fact as the *decomposition lemma*, which will play a very important role for our correctness proof for divergent programs. Nonetheless, our formalization so far is not capable of speaking about infinite execution traces, therefore an extension is ultimately necessary [7]. We will be utilizing ordinal numbers for this purpose.

5.1 Ordinal program states

Let α be an (possibly infinite) ordinal number; then, given any configuration, we have:

$$\langle P,\, \sigma,\, s\rangle \to^{\alpha} \langle P',\, \sigma^P_\alpha,\, s^P_\alpha\rangle$$

where $\to^\alpha$ means execution of the program α-many steps.

If α is a finite ordinal (i.e. a natural number), then we have already formalized what the transition above means. In the infinite ordinal case, σ^P_α is defined as follows:

$$\sigma^P_\alpha(x, s^P_\alpha) = \begin{cases} v & \text{if } v \in \mathbf{Val} \ \wedge\ \exists\beta < \alpha.\ \forall\gamma.\ \beta < \gamma < \alpha \implies \sigma^P_\gamma(x, s^P_\gamma) = v \\ \mu & \text{otherwise} \end{cases}$$

where μ is a special constant, denoting the value *unknown*. A reason for having the value unknown may be oscillation or indefinite growth/decrease. If the value of any variable x is *known* after executing α-many steps, then we say that x, *α-stabilizes*.

Moreover, since `SelVeri` allows a variable to be declared only once in a non-erroneous program, we need not extend the formalization of scopes. Throughout this section, we will restrict our attention on how the program states behave under infinite traces.

[7] Note that we will not be considering erroneous programs in this section, as they are finite by their nature.

5.2 Transfinite correctness of divergent programs

Now, as our formalization enhanced, we can attempt to prove the correctness of divergent `SelVeri` programs. More formally, we need to show that; given any configuration and an ordinal α

$$\langle P,\, \sigma,\, s\rangle \;\to^{\alpha}\; \langle P',\, \sigma^P_\alpha,\, s^P_\alpha\rangle \quad\Longrightarrow\quad \langle \mathcal{C}(P),\, e,\, \sigma,\, s,\, pc,\, ras\rangle \;\triangleright^{\beta}\; \langle \mathcal{C}(P'),\, e',\, \sigma^P_\alpha,\, s^P_\alpha,\, pc',\, ras'\rangle$$

where $\beta = k \cdot \alpha$ [8] for some constant non-zero $k \in \mathbb{N}$.

Proof. We will prove this by a transfinite induction on α. But before starting the proof; observe that the small-step operational semantics of `SelVeri` and `SelVerIR` are defined exclusively via successor transition relations, meaning that any execution trace is a sequence strictly indexed by the natural numbers, $\mathbb{N}$. Consequently, the execution trace of any divergent program has an exact ordinal length of ω, as the semantics do not define limit-ordinal transitions. Hence, we limit our attention to the ordinals that are lesser or equal to ω [9]. We call this fact as *ω-bound of computation*.

- **The Base Case:** $\alpha = 0$.
  Zero steps taken, the states are trivially identical.

- **The Successor Case:** Assume that theorem holds for some ordinal γ and let $\alpha = \gamma^+$.
  Section 4 exactly proves the successor case. Note that, the proof in the **section 4** trivially implies the prefix correctness via the semantics of compositional statements.

- **The Limit Case:** Let α be a limit ordinal and assume that the theorem holds for all ordinals $\gamma < \alpha$.
  By the *ω-bound of computation*, we have $\alpha = \beta = \omega$. This also means our induction hypothesis is equivalent to the correctness of convergent programs, which is already proven. Additionally, because the only reasons of divergence in a `SelVeri` program are infinite loops or recursions, after a finite number of steps, say M; by the decomposition lemma there must be an indefinitely executed statement S_r whose longest possible execution takes a finite number of steps, say m. Given any variable x, consider two cases:

  1. x ω-stabilizes.

     Then by the definition of stabilization, $\sigma^P_\omega(x, s^P_\omega) = v$ for some $v \in \mathbf{Val}$, meaning that there exists a natural number N such that for all natural numbers $n > N$, we have $\sigma^P_n(x, s^P_n) = v$. Now, let $K = max(N, M)$ and observe that for all k such that $K < k < \omega$, we have $\sigma^P_k(x, s^P k) = v$ since x is ω-stabilizing. Also, by our choice, the k^{th} step exactly lies in the indefinite repetition of S_r. This means that, after the step K; S_r (or any sub-statement of S_r) either never assigns any value to x or if it assigns, the value is strictly equal to v; otherwise the stabilization of x would be violated. In other words, we have reached to the fixed-point [10] of the value of x, under the semantics of S_r.
     Acknowledging the fact that S_r by itself is a finite program, $\mathcal{C}(S_r)$ does not modify the value of x after some step K' which is greater than K but still finite, as any compiled statement executes more (but still finite) steps than its high-level counterpart, by the correctness of finite compilation. This means that x is also an ω-stabilizing variable under the execution of $\mathcal{C}(P)$.

  2. x does not ω-stabilize. $(\sigma^P_\omega(x, s^P_\omega) = \mu)$

     Then, for an arbitrary finite step $N > M$, there exists some future step $n > N$ such that $\sigma^P_N(x, s^P_N) \neq \sigma^P_n(x, s^P_n)$. Let $i = \lceil (n - N)/m \rceil$ and let S^N_r be the composition of S_r by itself i-many times. (Similar to S_r, S^N_r is also executed indefinitely.) Then by construction, both the N^{th} and n^{th} steps of execution exactly lie in some indefinite repetition of S^N_r,

[8] Because the compilation function $\mathcal{C}$ emits a finite number of commands per high-level statements.
[9] We are not interested in hypercomputation here.
[10] We are slightly drifting into the denotational semantics region.

meaning that S_r^N does not preserve the value of the variable x. By its definition, one execution of S_r^N takes at most $i \cdot m$ steps, which is finite. By correctness of finite compilations (i.e. the induction hypothesis), $\mathcal{C}(S_r^N)$ also does not preserve the value of x.
Observe that with the exact same manner, for any sufficiently large N', we can construct a statement $S_r^{N'}$ (therefore a $\mathcal{C}(S_r^{N'})$ in the intermediate representation) that does not preserve the value of x, hence x also does not ω-stabilize under the execution of $\mathcal{C}(P)$, i.e. the value of x is μ in the IR counterpart as well.

These both cases ensure that the resulting program states after executing P and $\mathcal{C}(P)$ for ω-steps, are exactly equal; establishing the correctness of the compilation of divergent `SelVeri` programs.

□